Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Supporting Self-Descriptiveness, Reusability, and Code Bidirectionality in Spreadsheet Programming

Bachelor Thesis

Robin Frey

January 06, 2026

Advisors: Prof. Dr. Z. Su, Dr. S. Thorgeirsson
Department of Computer Science, ETH Zürich

Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

Abstract

Contents

Contents iii

1 Introduction 1
1.1 Motivation . . . 1
1.2 Problem statement . . . 1
1.3 Thesis outline . . . 2

2 Background 3
2.1 The spreadsheet paradigm . . . 3
2.1.1 Spreadsheet units . . . 3
2.1.2 Dependencies . . . 3
2.2 Cognitive dimensions . . . 4
2.3 Related work . . . 4

3 System Design 5
3.1 Architectural overview . . . 5
3.2 The presentation layer . . . 5
3.2.1 Layout strategy . . . 6
3.2.2 Flow graph justification . . . 6
3.2.3 Interaction design . . . 6
3.3 The visualization layer . . . 7
3.3.1 Literals . . . 7
3.3.2 Arithmetic operations . . . 8
3.3.3 Reference Nodes . . . 8
3.3.4 Functions . . . 10
3.3.5 Intermediate Results . . . 12
3.4 The parsing layer . . . 12
3.4.1 Spreadsheet grammar . . . 12
3.4.2 Transformers . . . 12
3.4.3 Serializer . . . 12

3.5 The data layer 12
3.5.1 Formula engine selection 12
3.5.2 Single engine instance 13
3.5.3 Asymmetric data flow 13
3.5.4 Data Structure 13

4 Evaluation **14**
4.1 Use cases 14
4.2 Study 14

5 Conclusion **15**

Bibliography **16**

Dummy Appendix **18**

Chapter 1

Introduction

1.1 Motivation

Spreadsheets are one of the most common computer tools, with an estimated 1 billion user base [3]. This popularity is largely attributed to their accessibility. Users can store and organize their data in near instant time without writing traditional code. This allows for quick "what-if" projections of their data for decision-making. [10]. Even though Spreadsheets do not require writing code, users still have to write formulas and possibly use multiple Spreadsheet at the same time. This hidden complexity can lead to weak points in the spreadsheets. Such possible weak points are called Spreadsheet smells. Common examples are Feature Envy, where a formula accumulates excessive dependencies, and Inappropriate Intimacy, when data is tightly coupled across different Spreadsheets [5]. These smells hinder comprehension and make it harder for the user to find the relevant information. Readers of Spreadsheet spend approximately 40% of their time "seeking information". This involves locating items, understanding interactions, tracing formula logic or understanding familiar spreadsheet concepts [9]. This shows that it exists a need for tools to help Spreadsheet users navigating and understanding the underlying complexity of Spreadsheets.

1.2 Problem statement

The study conducted by Srinivasa Ragavan suggests, that information detouring contributes to approximately 40% of the needed time to understand a Spreadsheet. In some cases participants even failed to obtain the relevant information. Information seeking involves identifying dependents of a cell, understanding the underlying logic of a certain value. Another time-consuming task is formula tracing. This occurs when a formula refers to another cell address, which again refers another and by doing so creating a

chain of dependencies, the rabbit hole effect. Although modern spreadsheet programs highlight referenced cells on the grid, it is not sufficient when the cells are not visible in the grid[9]. A second problem is the loss of context which occurs during the transition from the author to the reader. An author has implicit knowledge about provenance and relation of the data which rarely gets encoded in the spreadsheet. Consequently, the reader lacks the context to fully reconstruct the relationships and provenances [7]. These challenges highlight a gap in the current spreadsheet programs. While there is a clear need to improve the visualization for the logic of a Spreadsheet, visualization alone is not enough. To address the problem of information detouring a tool must actively support the user in the navigation process, to make it easier to find relevant information while minimizing the context loss of the process.

1.3 Thesis outline

Chapter 2

Background

2.1 The spreadsheet paradigm

2.1.1 Spreadsheet units

A spreadsheet program is used to organize, store and maintain data and perform calculations on the data. The primary unit of a spreadsheet is the cell, this is the place where data is stored. A cell can have three types of content: a static value, a formula or be empty. A value is data which is directly entered by the user. A formula is a recipe for calculating a value[2, p. 18]. For the thesis we define the terms worksheet and workbook as following:

Definition 2.1 *A **worksheet** is a two-dimensional grid of cells that are organized into rows and columns. [2, p. 65]*

Definition 2.2 *A **workbook** is a collection of worksheets. [2, p. 65]*

Some functions use cell ranges as arguments. A cell range specifies a region of cells.

2.1.2 Dependencies

If a formula references other cells it creates dependencies.These referenced cells are the dependents of the cell where the formula resides:

Definition 2.3 ***Dependents** of a cell address or range are :*

1. *All cells with formulas that contain the given cell address or range.*
2. *Some of the ranges that contain the given cell address or range.*

[1]

The dependencies can be tracked and modeled as a Directed Graph. With the constraint that Spreadsheets do not allow cyclic referencing, the resulting

graph will be an Acyclic-Directed Graph. But spreadsheets programs do not explicitly show the dependency graph and the dependents stay hidden. Such hidden dependencies can cause hidden changes. For example if we have a formula in one worksheet, which is dependent on a cell in another worksheet, and change the data in this cell the change in the computed value of the formula stays hidden [4]. The change stays hidden because almost all Spreadsheets only display one worksheet at a time.

2.2 Cognitive dimensions

In order to describe and explain certain design choices in more detail the thesis uses definitions from the "Cognitive Dimensions" Framework [4].

(Here I will write about for example diffuseness, viscosity, consistency, hidden dependencies. And formally define it according to the Paper to use it for reasoning about certain design choices. For example Diffuseness, viscosity, role expressiveness)

2.3 Related work

(Here I want to mention similar tools which also tried to solve these problems especially I will mention the already existing arrows for dependencies in excel, fluid visualization paper, Hermans generating leveld dataflow paper)

Chapter 3

System Design

3.1 Architectural overview

The system can be divided into four layers.

1. The presentation layer
2. The visualization layer
3. The parsing layer
4. The data layer

The **presentation** Layer is responsible for interactions and rendering. It implements a split-view displaying the datatable and flow-graph simultaneously. This layer only presents the data from the state layer and dispatch ssers interactions. It does not directly manipulate the data. The second layer, the **visualization layer**, is responsible for bridging the gap between the underlying formula structure and the flow graph visualization. The **parsing** layers only responsibility is to provide two conversions one from formula strings into an AST and the reversed direction. The foundation of the system is the **data layer** where the calculation engine of the spreadsheet resides and manages all the spreadsheet data.

3.2 The presentation layer

For the presentation layer there are 3 important goals:

1. improve self-descriptiveness of the hidden data of the datatable
2. improve bidirectionality between the tool and the datatable
3. minimize the cognitive load of the user

3.2.1 Layout strategy

Placing the datatable-view and flow-graph-view side by side serves two purposes. First by having them as near as possible to each other it should reduce the cognitive distance between datatable-view and flow-graph-view. Secondly by strictly separating the both views i.e not having a popover or even an overlay strategy, prevents the occlusion of other important cells.

3.2.2 Flow graph justification

The decision to represent the hidden data as a flow graph was driven by the nature of the underlying state of spreadsheet formulas, the dependency graph. A flow graph allows us to uncover this state and enables the user to see the dependencies rather than by reading formulas and parsing it on their own. The flow direction is left to right. This should mirror the spatial convention for spreadsheets, where the inputs of a formula are located in columns on the left side of the formula.

3.2.3 Interaction design

The flow graph not only serves as a visualization but also as an interface for the exploration and manipulation of the datatable. Interactions are organized into two distinct modes: the **preview** mode and the **edit** mode. This separation should provide clear visual feedback when changes will affect the underlying data of the datatable.

Preview Mode

The Preview Mode is the default state and is intended for the exploration. Hovering over a reference node highlights the corresponding cell or range in the datatable. This allows the user to survey the dependency at the current position in the datatable. Clicking on a reference node scrolls the datatable to the corresponding cell or range to enable closer inspection of the dependencies.

Edit Mode

Edit Mode is always entered through an explicit user action:

1. clicking on the edit button
2. double-clicking on an inline value or address
3. double-clicking an edge
4. dragging from a connection handle

The entry action determines the initial editing context. Double-clicking on a node's address allows the user to select a new address directly from the datatable. The change is committed by pressing enter or clicking the confirmation symbol. Double-clicking an inline value transforms the field into an editable input. The user modifies the constant and commits the change through the same confirmation mechanism. Double-clicking an edge enters the edit mode with the double-clicked edge selected for further manipulations. Dragging from a connection handle initiates edge creation or modification. The user drags from a source handle to a target handle. While dragging the edge invalid handles are grayed out. If the target handle has no existing connection, the new edge connects directly. Else the target handle already has a connection, a popover asks the user to either replace the existing edge or swap positions with it. Clicking only the edit button enters the edit mode without an initial editing context. Once in edit mode, the user can change context freely.

3.3 The visualization layer

In the visualization layer the system transforms the AST into a graph representation which is then rendered as an interactive flow graph. This section focuses on the visual design choices of the flow graph nodes and how each AST node maps to a corresponding graphical representation. The primary goal when transforming the AST is to reduce the overall number of nodes and edges while improving clarity. This aims to create a compact and informative flow graph. Literal nodes are inlined into their parent nodes. Arithmetic operations are either transformed into a binary operation node or collapsed together into one expression node. Reference nodes are enhanced with visual cues and header inference heuristics. Function nodes are augmented with semantic argument labels. Conditional functions are represented by a separate node to help the comprehension of the branching logic. Function nodes as well as reference nodes display their intermediate results to provide more information about the current state to the user.

3.3.1 Literals

Literals are for example numbers or strings, literals are used almost in every formula. Because the meaning of literals are evident rendering them as nodes would node add meaningful help for the users. The system solves this by inlining literals into their parent node. There they are still visible but do not occupy any additional space in the graph.

3.3.2 Arithmetic operations

3.3.3 Reference Nodes

In order to understand the dependencies of the formula the system explicitly creates nodes for references. A reference node has three important components: Address, label and result. The most important component is the address component. The address is colored accordingly to the pre-defined color of its worksheet. When importing a new workbook the system generates distinct colors for each worksheet. The coloring gives a fast visual cue to the users where nodes reside. For example when formulas refer across worksheets two distinct addresses can look the same even though they refer two different cells or ranges (see Figure 3.1).

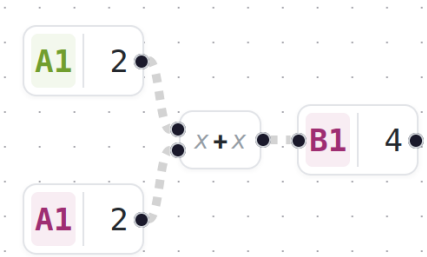

Figure 3.1: First image

Figure 3.2: Second image

Label

Hodnigg et al. identify the comprehension indicator "realization" which refers to an insight of the spreadsheet to the user. And one of the comprehension patterns is reading a label[1] followed by a "realization". This indicates that labels can help in the process of comprehending spreadsheets. To help the user in the process of reading labels, the system implements a header inference heuristic. One limitation of this heuristic is that it assumes well-structured spreadsheets and may yield suboptimal results in spreadsheets that do not follow standard column/row orientation. To generate a label the system scans upwards and leftwards and selects the first non-formula string value to compose them into a composite label. For example in Figure 3.3 the system resolved J4 to Bob:Rounded. By inferring the label for the user, the system aims to preserve the user's spatial context within the data table, as the users are no longer required to visually scan through columns and rows.

[1] textual descriptions in cells

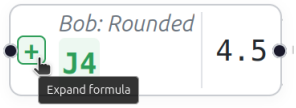

Figure 3.3: Expandable cell reference node when minimized

Figure 3.4: Expandable cell reference node when expanded

Expandable Cell References

As mentioned in section 1.2, one major challenge for spreadsheet comprehension is the tracing of the dependency chain. To address this the system implements a recursive resolution strategy, to build the dependency chain.

Definition 3.1 *An expandable cell reference is a precedent that itself contains a formula*

To not overwhelm a user when presented the whole dependency chain of the synchronized cell, the system adopts a details-on-demand [8] strategy. Upon synchronizing a new cell, the expandable cell reference nodes start in a minimized state (see fig. 3.3). Expandable cell reference nodes (see fig. 3.4) are only expanded upon explicit user interaction. With this strategy the system aims to reduce the Diffuseness of the flow graph. Crucially the graph navigation is decoupled from the grid navigation. When a user expands a cell reference node the viewport of the worksheet remains the same. This aims to preserve the spatial context of the user in the worksheet while being able to trace down the dependency chain, directly addressing an issue reported by Srinivasa et al., where a user has lost track when navigating between dependents and precedents [9].

Merging references

In Spreadsheets it is common that formulas use the same references. The flow graph tries to merge same reference nodes, which are located in a distance of 5 edges, into one single node. The distance of 5 edges was chosen arbitrarily but by merging same reference nodes together the system aims to make the flow graph more compact and less confusing.

3.3.4 Functions

A function in Spreadsheet programs is a predefined formula, which takes one or more arguments to produce a result [2]. In traditional formula syntax, the semantic meaning of an argument is only defined by its position for example in the function: VLOOKUP(E6,H6:I9,2,0). When reading this the user must actively recall which argument is responsible for what. This mental mapping increases cognitive load. To solve this the system gives each argument a semantic label. By visually linking a node to a semantic label the system aims to increase the role expressiveness of the argument nodes. This should decrease the need of consulting a documentation.

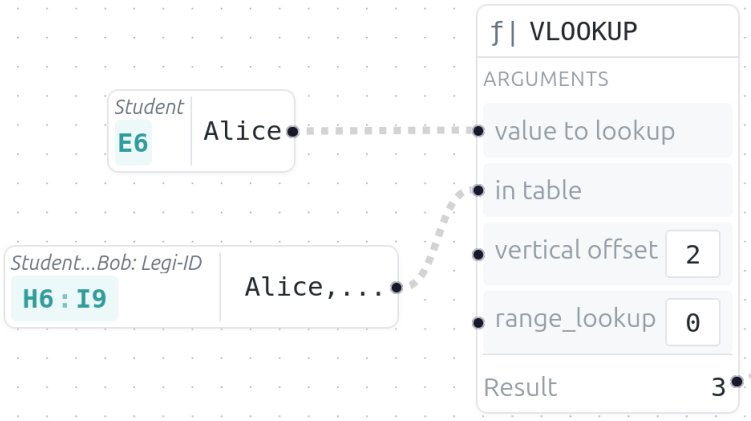

Figure 3.5: Example of the linking between argument nodes to semantic argument labels

Conditional Nodes

Conditional functions[2] introduce branching logic to Spreadsheets and add additional challenges for formula tracing. First, as mentioned in the section above 3.3.4, conditional arguments also suffer from the positional semantics problem. Furthermore, conditional functions in textual form suffer from the lack of runtime visibility. Users must mentally simulate the boolean condition to determine which branch is active. To address this, the system only displays the currently active branch and minimizes the inactive branches at first. This also helps to reduce the diffuseness of the flow graph but hides some logic as tradeoff. To highlight the active branch to the user, the system implements the following strategies:

1. Minimization of inactive branches
2. Visual dimming

[2] e.g IF, IFS

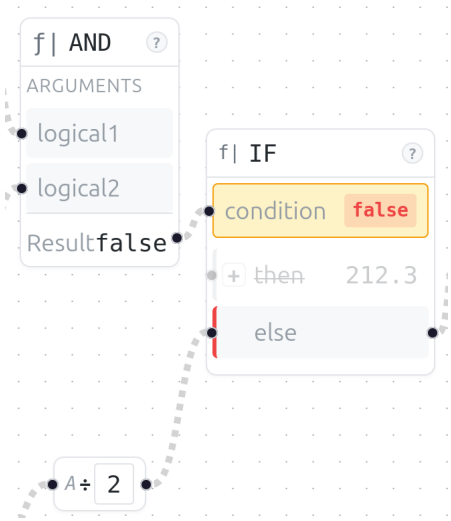

Figure 3.6: The system strikes non taken arguments in conditional nodes through

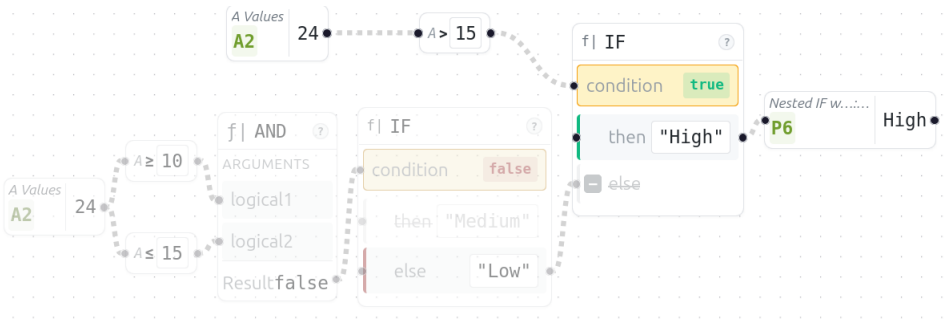

Figure 3.7: The system grays non taken branches out if expanded

3. Highlighting the contributing argument

As mentioned above the system minimizes inactive branches by default. If the users still want to explore the logic of the inactive branch they can expand the subflow by clicking on the plus symbol. This mechanic is identical to the one of expandable cell references as mentioned in section 3.3.3. In order to still have a visual difference between the active and inactive branches, when the user expands inactive branches, the system dims the inactive branches (see fig 3.7). And to reinforce this state in the node itself the inactive arguments are struck through (see fig 3.6). These visual cues aim to prevent misinterpreting an inactive value as contributing factor to the result of the conditional node. Furthermore, because spreadsheets are dynamic the system updates the active branch when dependents change.

3.3.5 Intermediate Results

Another comprehension pattern Hodnigg et al. identify is that after reading values realizations can follow [6]. This is one of the reasons why the system displays the intermediate results of reference nodes or functions. Intermediate results in reference nodes refer to the content of the cell or range while in function nodes to the calculated output of the function. The second reason the system explicitly shows the intermediate results is to allow the users to verify each sub-step in the Flow Graph.

3.4 The parsing layer

3.4.1 Spreadsheet grammar

(Here I will write about the Spreadsheet grammar ands its detail and also list the detailed grammar used in the parser)

3.4.2 Transformers

(Here I will write about how the system transforms the AST when we have edited something. The transformers are cell reference transformer, literal transformer, expression replacement transformer and also how the system swaps or adds function arguments)

3.4.3 Serializer

(Here I will write about how the system serializes the AST back into a formula. Maybe I will also make a diagram to show the flow of these)

3.5 The data layer

The data layer forms the foundation of the system architecture. It serves as the single source of truth for all spreadsheet related data. This includes for example: cell values, formulas and computed results.

3.5.1 Formula engine selection

For the formula engine it is important that it provides a lot of functions which are also used in Microsoft Excel or Google Sheets to make the user as familiar with the formulas as possible. Another important feature should be the ability to handle multiple worksheets. The system uses HyperFormula as its calculation engine. HyperFormula provides compatible syntax with Microsoft Excel, 400 built-in functions and an easy integration with custom front-ends [1]. This allows the user to transfer their pre-knowledge about spreadsheet functions and behavior to this system.

3.5.2 Single engine instance

A single HyperFormula instance maintains the whole calculation state for the entire workbook. This allows the user to reference cells and ranges from different worksheets. Having only one single-instance implies having only one dependency graph for all worksheets. This simplifies the process of checking and updating dependencies. But this also implies that one computationally heavy formula could affect the responsiveness of the entire workbook.

3.5.3 Asymmetric data flow

As mentioned in the presentation layer 3.2 each worksheet is implemented by their own Handsontable component. Each component reads/writes from/to the same Hyperformula Engine. All components of the Graph-View also read directly from the same Hyperformula Engine. A read is done for example when nodes calculate their intermediate results. In contrast to the read paths, data manipulation in the Graph-View follows a slightly different path. Because HyperFormula is a headless calculation engine it is not able to know if it is connected to any Handsontable component. Therefore to prevent inconsistencies between the datatable and the underlying calculation engine, edits to the data have to go through the active Handsontable component to trigger rendering and the corresponding hooks.

3.5.4 Data Structure

(Here I will write about the underlying data of the engine what type of data does HyperFormula recognize / store)

Chapter 4

Evaluation

4.1 Use cases

(Maybe I will write about for what type of user the tool could be helpful)

4.2 Study

(Here I will write everything about the small study from what goals it has, what type of questions and tasks I asked and study conclusion)

Chapter 5

Conclusion

Bibliography

[1] HyperFormula (v3.1.1).

[2] ECMA-376-1:2016 Office Open XML File Formats - Fundamentals and Markup Language Reference, 2016.

[3] Hjalmar Gislason. Excel vs. Google Sheets usage — nature and numbers, January 2021.

[4] T.R.G. Green and M. Petre. Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework. *Journal of Visual Languages & Computing*, 7(2):131–174, June 1996.

[5] Felienne Hermans, Martin Pinzger, and Arie van Deursen. Detecting and visualizing inter-worksheet smells in spreadsheets. In *2012 34th International Conference on Software Engineering (ICSE)*, pages 441–451, June 2012.

[6] Karin Hodnigg, Christian Macho, Martin Pinzger, and Dietmar Jannach. Comprehending Spreadsheets: Which Strategies do Users Apply? In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, pages 386–390, May 2021.

[7] Andrea Kohlhase, Michael Kohlhase, and Ana Guseva. Context in Spreadsheet Comprehension.

[8] Ben Shneiderman. The eyes have it: A task by data type taxonomy for information visualizations. In *The craft of information visualization*, pages 364–371. Elsevier, 2003.

[9] Sruti Srinivasa Ragavan, Advait Sarkar, and Andrew D Gordon. Spreadsheet Comprehension: Guesswork, Giving Up and Going Back to the

Author. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–21, Yokohama Japan, May 2021. ACM.

[10] Melissa Rodriguez Zynda. The first killer app: a history of spreadsheets. *interactions*, 20(5):68–72, September 2013.

Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).